

속도는벡터 — 엔진 적용 검증 실험 정리 (PostgreSQL 재현용)

목적: 임채림 연구원님의 PostgreSQL 재현·검증 및 SAP 미팅 참고 자료. **작성:** 2026-05-22 · 속도는벡터 (연세대 캡스톤, 지도 박광현 교수님) **핵심 한 줄:** 오프라인에서 검증한 *결합 카디널리티 추정값*을 Exqutor-패치 PostgreSQL에 주입했을 때, 추정 정확도가 **실행 계획의 견고한 회복**과 **end-to-end 실행 시간**으로 이어짐을 sf=10 4개 데이터셋(DEEP·SIFT·SSN·YFCC)에서 측정으로 확인하였다.

이 문서는 **재현 명세**다 — 결과 요약뿐 아니라, 같은 실험을 PostgreSQL에서 다시 돌리는 데 필요한 환경·파라미터·절차·함정을 모두 담는다.

1. 연구 개요 (배경을 모르는 분도 이해할 수 있도록)

1.1 문제 — 벡터 증강 분석 쿼리(VAQ)의 카디널리티 추정

분석 DB는 쿼리를 실행하기 전에 "이 조건을 만족하는 행이 몇 개일까"(카디널리티)를 추정해 실행 계획을 고른다. 추정이 빗나가면 옵티마이저가 엉뚱한 계획(예: 불필요한 Hash Join, 잘못된 조인 순서)을 골라 쿼리가 느려진다.

벡터 술어(`embedding <-> query < threshold` 형태의 유사도 범위 조건)가 섞인 쿼리 — **VAQ** — 에서는 이 추정이 특히 어렵다. Exqutor 논문(arXiv:2512.09695v2)은 기존 엔진이 벡터 술어에 **고정 비율**(pgvector 33.3%·VBASE 50%·DuckDB 100%)을 쓰다 잘못된 계획을 낸다고 지적하고, 인덱스가 없을 때 **적응적 표본 추출(Adaptive Sampling, 논문 §V-B)**로 카디널리티를 추정하는 방법을 제안한다.

1.2 본 연구의 개입 — 표본 선택 단계 하나

Exqutor의 적응적 표본 추출은 **무작위 베르누이(Bernoulli) 표본 추출** — 모든 행을 같은 확률로 독립 추출 — 로 표본을 뽑는다. 이 방식은 데이터가 고르게 퍼져 있을 때는 잘 작동하지만, **실제 벡터 데이터는 특정 영역에 행이 쏠려 있다(skew)**. 쏠린 분포에서 무작위 추출은 밀집 구역을 표본에 충분히 담지 못해 추정량의 분산이 커지고, 같은 쿼리라도 추정값이 들쭉날쭉해진다 — 추정이 흔들리면 옵티마이저가 잘못된 계획을 고를 위험도 커진다. **본 연구의 출발점이 바로 이 지점이다.**

본 연구는 이 **표본 선택 단계 하나**만 — 무작위 베르누이 → **데이터 분포를 인지한 층화(stratified) 표본 추출** — 로 바꾸는 개입의 효과를 검증한다. 층화 추출은 데이터를 비슷한 성질의 계층(stratum)으로 먼저 나눈 뒤 각 계층에서 표본을 뽑아, 쏠린 영역도 표본에 빠짐없이 대표되게 한다. 논문의 표본 예산(N=385)·식 1-6은 그대로 둔 채, "같은 예산으로 표본을 어느 계층에서 뽑을지"만 바꾼다.

검증은 세 갈래로 짝지어 측정한다(3-way matched).

약칭	의미	카디널리티 추정 방식
B1	대조군 (논문 그대로)	무작위 베르누이 표본 추정값 <code>est_b1</code>
CaseA	음성 대조군 (완전 대체)	분포 인지 표본 추정값으로 통째 치환 (<code>est_method</code> 단독)
CaseB	결합 (제안)	두 추정값의 산술 평균 <code>est_final = (est_b1 + est_method) / 2</code>

세 갈래를 함께 측정하는 이유는, 개선이 **분포 인지 표본 자체**에서 오는지 아니면 **두 방식의 결합**에서 오는지를 가려내기 위함이다 — CaseA(완전 대체)는 음성 대조군 역할을 한다.

오프라인 카디널리티 추정 측정(RQ3, 16종 method × 5 데이터셋 × 5 조작변인, 3-way 1,508건)에서, 결합(CaseB)은 대조군 B1 대비 약 9할의 비교에서 Q-error를 낮췄고, 완전 대체(CaseA)는 불안정했다 → "**기존 방식을 대체하기보다 보완·결합하는 것이 효과적**"이라는 결론.

1.3 "엔진 적용 검증"의 의미 — 이 문서가 다루는 실험

오프라인 추정 측정은 "추정값이 더 정확하냐"까지만 답한다. **엔진 적용 검증**은 그 다음을 묻는다 — 검증된 결합 추정값을 **실제 PostgreSQL 엔진에 주입**했을 때, 추정 정확도가 (a) 옵티마이저의 **실행 계획 결정**과 (b) **end-to-end 실행 시간**으로 이어지는가? 본

문서의 측정이 바로 이 부분이며, "추정 → 계획 → 실행 시간"의 인과 사슬을 닫는 단계다.

한 줄 줄거리: Exqutor의 무작위 베르누이 표본 추출은 쏠린 데이터에서 추정이 불안정하다 → 분포 인지 총화를 베르누이와 결합하면 추정이 안정된다 → 그 결합 추정값을 엔진에 주입하면 옵티마이저가 **정답 실행 계획을 견고하게 회복**한다.

2. 실험 환경

2.1 서버

항목	값
호스트	165.132.140.240 (BDAI 연구실 서버, 계정 capstone2026)
작업 디렉토리	/mnt/hdd0/home/capstone2026/
하드웨어	Intel Xeon Gold 6530 · 128 vCPU · 1.0 TB RAM · 4× NVIDIA RTX 6000 Ada (측정은 CPU만 사용)

2.2 patched PostgreSQL — Exqutor 패치 pgvector

측정 대상 엔진은 Exqutor 논문 방식으로 패치한 pgvector 확장을 올린 PostgreSQL 인스턴스다.

항목	값
포트	55435 (소켓 /tmp)
DB / 사용자	wns41559 / wns41559
패치 핵심	벡터 술어 카디널리티를 외부에서 주입할 수 있는 GUC 3종

주입용 GUC (vector 확장이 정의 — 세션 첫 statement로 **LOAD 'vector'** 필요):

GUC	역할
vector.injected_card	이 값(≥0)을 벡터 술어 카디널리티로 강제 사용 (샘플링 건너뛰)
vector.disable_estimation	on 이면 추정 자체를 끄고 플래너 default selectivity 사용
vector.update_sample_size	표본 크기 동적 갱신 — 본 측정에서는 전 조건 off 고정

2.3 데이터 — 벡터 테이블

서버 테이블은 **partsupp_<데이터셋약칭>_<sf>** 명명규칙을 따른다. 데이터셋명과 테이블 약칭이 다른 경우가 있으니 주의 — 특히 SSN.

데이터셋	차원	테이블 약칭	sf=10 테이블명
DEEP	96	deep	partsupp_deep_10
SIFT	128	sift	partsupp_sift_10
SSN (SimSearchNet++)	256	fb ⚠	partsupp_fb_10
WIKI	768	wiki	partsupp_wiki_10
YFCC	192	yfcc	partsupp_yfcc_10

⚠ **SSN ↔ fb alias:** 논문·문서 표기는 SimSearchNet++(SSN)이나 서버 테이블·query_pool 파일은 fb (Facebook AI 출처) 약칭을 쓴다. 재현 시 SSN을 측정하려면 테이블 **partsupp_fb_10** 을 본다.

VAQ 쿼리는 TPC-H 기반이므로 TPC-H base 7종(**customer.orders.lineitem.supplier.nation.region.part**)도 sf별로 적재돼 있다(**<테이블>_<sf>**).

2.4 측정 하네스

_internal/scripts/measure_latency_realengine.py (Python, **psycopp** 사용). 한 cell(쿼리×데이터셋×sf×selectivity×질의벡터)을 받아 16개 variant를 측정하고 JSON 하나를 산출한다. 하네스가 처리하는 환경 준비:

- 세션마다 `LOAD 'auto_explain'` → `LOAD 'vector'` 순서 고정 (역순이면 Exqutor 2-pass의 pass-2 플랜이 로깅되지 않음 — §4.1).
- `partsupp_deep` + base 7종 이름을 SF 테이블로 잇는 임시 VIEW를 만들어 VAQ SQL 템플릿을 무수정 실행 (세션 종료 시 자동 소멸 → 타 사용자 무영향).
- `work_mem = 256MB`, `max_parallel_workers_per_gather = 2`, `plan_cache_mode = force_custom_plan`.

3. 측정 평면·파라미터

3.1 측정 평면 (sf=10)

변인	값
Scale Factor	<code>sf=10</code> (partsupp ≈ 8천만 행)
데이터셋	DEEP · SIFT · SSN · YFCC (WIKI는 §5.3 예외)
쿼리	TPC-H <code>q3</code> · <code>q9</code> · <code>q10</code> · <code>q12</code> (벡터 솔어가 SeqScan 분기에서 평가되는 분석 쿼리)
selectivity	<code>0.001</code> · <code>0.01</code> · <code>0.1</code> (벡터 솔어가 통과시키는 행 비율)
질의 벡터 id	qid 0 (확장 평면) / DEEP 정본은 qid 0·1·2

측정 단위(cell) = 쿼리 × 데이터셋 × sf × selectivity × qid 하나. 각 cell마다 16 variant를 측정한다.

3.2 한 cell의 16 variant — 4 조건

조건	variant 수	주입 GUC	의미
baseline (기본엔진)	1	<code>disable_estimation=on</code>	카디널리티 개입 없음 — 플래너 default
B1 (베이스라인 추정)	1	<code>injected_card = est_b1</code>	논문 무작위 베르누이 추정
oracle (정답)	1	<code>injected_card = true_card</code>	참 카디널리티 — 이론적 상한
CaseB (결합 추정)	13	<code>injected_card = est_caseB</code>	결합 추정 — 강한 13 method 각각

CaseB 13 method (오프라인 16종 중 불안정 클러스터링 3종 gmm·minibatch_partial·faiss_ivf 제외): `hilbert_real` · `skilling_hilbert` · `chao_weighted` · `ica_fastica` · `pca1d` · `zorder_morton` · `hyperloglog` · `cum_sqrtf` · `lavallee_hidiroglou` · `rsvd` · `sparse_rp` · `mhst2` · `rabbitq_strat`

★ **latency layer = 4 조건** (baseline/B1/oracle/CaseB)이며 CaseA(완전 대체)는 latency 측정에 포함하지 않는다 — 오프라인 추정 연구의 3-way(B1/CaseA/CaseB)와 layer가 다르다(의도된 설계).

3.3 반복 측정

- 한 cell = (1 warmup + 15 timed) rep. 매 rep마다 16 variant 순서를 무작위 셔플(고정 seed) → 같은 rep 안에서 모든 variant가 인접 시점에 측정돼 짝지은 비교(matched)에 정합.
- 쿼리는 EXPLAIN이 아니라 실제 실행 + 결과 fetch까지 `perf_counter` 로 측정 (2-pass 오버헤드 포함 정직한 end-to-end 값).
- `statement_timeout = 180s` — 도달 시 해당 측정은 None(검열)으로 기록.

3.4 측정 완결 현황

데이터셋	측정 cell (JSON 산출)	분석(paired) cell	비고
DEEP	20 (정본)	20	phase2: sel=0.001 12 cell + phase3: sel=0.01·0.1 carry 8 cell
SIFT	12 / 12	12	q3·q9·q10·q12 × sel 0.001·0.01·0.1 — 전 cell 주입 발동
SSN	12 / 12	9	q3·q9·q10 발동 / q12 ×3은 주입 미발동
YFCC	12 / 12	9	q3·q9·q10 발동 / q12 ×3은 주입 미발동
WIKI	0 / 12	0	측정 불가 — §5.3 관측된 예외

⚠️ "측정"과 "분석"의 구분: SSN·YFCC의 q12 cell은 측정(JSON 산출)은 됐으나, 옵티마이저가 해당 벡터 테이블을 IndexScan 경로로 풀어 §4.1의 카디널리티 주입이 발동하지 않았다(`injection_fired=false`) → 짝지은 비교에서 자연 제외된다. 따라서 신규 3종 (SIFT·SSN·YFCC)의 분석 대상 paired cell = SIFT 12 + SSN 9 + YFCC 9 = 30 cell이다. 재현 시 SSN/YFCC q12를 대조하면 주입이 발동하지 않아 paired 결과가 나오지 않는 것이 정상이다.

4. 방법론 — 주입·계획 캡처·통계

4.1 카디널리티 주입 메커니즘 (Exqutor 2-pass)

Exqutor 패치는 실행 시점 2-pass 구조다.

1. pass-1: `planner_hook` 이 기본 selectivity로 1차 플랜을 만든다.
2. `ExecutorRun` 혹은 벡터 술어를 탐지하면 `check_for_vector_search` 가 카디널리티를 확정한다. `vector.injected_card ≥ 0` 이면 샘플링을 건너뛰고 그 값을 쓴다.
3. pass-2: 확정된 카디널리티로 재plan → 재실행. 측정한 latency는 이 2-pass 전체 시간이다.

⚠️ 주입은 벡터 테이블이 pass-1 플랜에서 `SeqScan` 으로 접근될 때만 발동한다 (`check_for_vector_search` 의 SeqScan 분기). 테이블이 작아 pkey IndexScan 경로가 선택되면 주입이 우회된다(예: sf=1 — §5.3). 그래서 측정마다 Exqutor의 `Estimated cardinality for range query...` 로그를 포착해 `injection_fired` 플래그로 주입 발동을 검증한다.

`LOAD 'auto_explain'` 을 `LOAD 'vector'` 보다 먼저 해야 한다 — 역순이면 pgvector의 `ExecutorEnd` 혹은 체인이 `auto_explain` 을 우회해 pass-2 플랜이 로깅되지 않는다.

4.2 실행 계획 캡처

`auto_explain` (JSON)으로 pass-2 실행 플랜을 받아, 각 variant의 플랜을 `plan_signature` — 플랜 노드 타입(Node Type)의 pre-order 튜플 — 로 압축한다. oracle variant의 `plan_signature`를 기준으로, 각 variant가 같은 계획을 만드느지로 "정답 계획 회복" 여부를 판정한다.

4.3 통계 파이프라인

산출 JSON → `analyze_latency.py` (cell별 paired 통계) → `stats_poc_6_4_extended.py` (3 평면 통합 분석). 짝지은 비교에 적용하는 검정:

- paired Wilcoxon 부호순위 검정 + Holm 다중비교 보정
- 효과크기 Hedges' g (소표본 보정), matched-pairs rank-biserial
- cluster paired bootstrap (cell 단위 재추출 — within-cell 상관 보정, B=2,000)
- 변동 분해(variance decomposition) — `log(exec_ms)` 를 query·qid·sel·condition·교호작용으로 분해

5. 결과

5.1 DEEP sf=10 핵심 평면 — sel=0.001 12 cell (phase2)

DEEP sf=10의 핵심 검증 평면은 selectivity=0.001 12 cell(측정 단계 phase2)이며, 아래 carry 수치는 이 12-cell core 기준이다. sel=0.01·0.1 8 cell(phase3)을 더한 20 cell이 DEEP sf=10 정보 평면 전체이고, 20-cell 전체의 pair 단위 회복률(93.2%, 280 paired)은 §5.2 데이터셋 표를 참조한다.

결과	정량	의미
기본엔진 대비 가속 (oracle, 12 cell 평균)	5.67× (q3 7.40× ~ q9 2.93×)	추정 정확도가 실행 시간으로 변환됨
B1 정답 계획 회복 (cell 단위)	7 / 12 = 58.3%	베르누이는 질의 벡터에 따라 취약 (qid0 0/4 · qid1 4/4 · qid2 3/4)
결합 13종 정답 계획 회복 (method×cell)	148 / 156 = 94.9%	결합은 결정 경계 근처에서도 견고 (실패 8건 모두 q3)
paired Wilcoxon, 기본엔진 대비 (anchor=baseline)	180 / 180 = 100% large effect	가속의 통계적 견고성
paired Wilcoxon, 결합 vs B1 (anchor=B1)	13 / 168 = 7.7%만 유의 · 86.9% small effect	결합·정답·B1의 실행 시간은 동등

★ 위 carry 수치(5.67×·7/12·148/156·180/180·13/168)는 DEEP sf=10 phase2 분석의 산출로 6/11 최종보고서 §5에 정본으로 실려 있다 — §5.2의 poc_6_4_extended 36-cell 재집계와는 별개 측정·집계 단계다.

해석: 결합 13종은 베이스라인보다 더 빨라지는 것이 아니라 더 안정적으로 정답 계획에 수렴한다. 실행 시간 자체는 세 주입 조건 (B1·CaseB·oracle)이 동등하면서, 계획 회복의 견고성에서만 우위를 갖는 — 두 결론이 분리되어 나온다.

5.2 sf=10 데이터셋 일반화 (신규 측정, 36 cell)

DEEP 외 SIFT·SSN·YFCC sf=10을 측정해 위 결론이 데이터셋 수준에서 일반화되는지 검증했다. (anchor=B1 — 베르누이 추정 대비 결합 추정의 짝지은 비교.)

데이터셋별 (sf=10)

데이터셋	paired n	정답 계획 회복률	평균 Hedges g	large effect 비율
DEEP	280	93.2%	0.336	6.8%
SIFT	168	94.6%	0.203	0.0%
SSN	126	93.7%	0.226	0.8%
YFCC	126	88.1%	0.199	0.0%

신규 3종 통합 (phase4_extension, SIFT+SSN+YFCC, 420 paired): 정답 계획 회복 92.4% · 평균 |g| 0.209 · 효과크기 94.3%가 small · Holm 보정 후 유의 0.0%.

변동 분해 (4 데이터셋 통합, B1·CaseB·oracle 3조건 모델 — 기본엔진 제외, n=2,250): 실행 시간 변동을 selectivity 49.1% · qid 28.0% · query 5.5% · 잔차 17.3%가 설명하고, 세 주입 추정(B1·CaseB·oracle) 중 어느 것을 주입했는가의 설명력 = 0.00% (Type-III p=0.945).

⚠ 이 0.00%는 기본엔진(baseline)까지 동등하다는 뜻이 아니다. 기본엔진은 카디널리티 주입 자체가 없어 느리며(§5.1의 5.67× 가속이 그 차이), 기본엔진을 포함한 4조건 모델에서는 condition 설명력이 약 21%다. 즉 두 결론이 분리된다 — "주입하느냐 마느냐"는 크게 다르고(5.67× 가속), "어느 추정을 주입하느냐"는 동등하다. 결합(CaseB)의 가치는 실행 시간 추가 단축이 아니라, 정답 계획을 견고하게 회복하는 안정성에 있다.

결론: DEEP sf=10에서 본 패턴 — 정답 계획의 견고한 회복 + 실행 시간 동등성 — 이 SIFT·SSN·YFCC로 데이터셋 수준에서 재현된다. 가장 까다로운 cell(q3 selectivity=0.1)까지 채운 뒤에도 회복률이 떨어지지 않았다.

5.3 관측된 예외 (honest exception)

- WIKI sf=10 — 측정 불가: WIKI는 768차원으로, 인덱스 없는 SeqScan이 statement_timeout (180초)을 초과한다. 측정을 시도했으나 12 cell 모두 시간 초과로 산출되지 않았다. 고차원 비인덱스 SeqScan이 실용 측정 시간을 넘어선다는 것 자체가 관측된 한계다.
- sf=1 — plan-invariant: 작은 테이블에서는 옵티마이저가 IndexScan 경로를 골라 SeqScan 분기로 진입하지 않으므로 주입이 발동하지 않는다(§4.1). plan-invariant 예외로 분리.
- sf=100 · 다른 엔진(VBASE·DuckDB·Milvus) = future work.

6. PostgreSQL 재현 절차

6.1 환경 준비

1. Exqutor 방식으로 패치한 pgvector 확장을 PostgreSQL에 빌드·설치. `vector.injected_card` · `vector.disable_estimation` · `vector.update_sample_size` GUC가 인식되는지 확인.
2. 측정할 데이터셋의 벡터 테이블(`partsupp_<약칭>_<sf>`)과 TPC-H base 7종(`<테이블>_<sf>`) 적재.
3. 세션에서 `LOAD 'vector'` 후 `SHOW vector.injected_card` 로 GUC 인식 확인.

6.2 추정치 준비

`_internal/scripts/gen_latency_estimates.py` 로 cell별 추정치 parquet(`estimates_<데이터셋>_sf<sf>.parquet`)를 생성한다. 컬럼: `dataset·sf·sel·query_id·method·qvec·D·true_card·est_b1·est_caseB` . — 이미 산출된 parquet은 `_internal/cache/rq3/latency/phase4_extension/` 에 있다.

6.3 하네스 실행

```
python3 _internal/scripts/measure_latency_realengine.py \  
  --query q3 --dataset DEEP --sf 10 --sel 0.01 --query-id 0 \  
  --estimates _internal/cache/rq3/latency/phase4_extension/estimates_DEEP_sf10.parquet \  
  --output _internal/cache/rq3/latency/<출력디렉토리> \  
  --statement-timeout 180s
```

서버 미접속 상태에서 SQL 변환·GUC·임시 VIEW DDL만 검증하려면 `--dry-run` . 한 cell ~ 정상 차원(128-256d)에서 약 35-40분.

6.4 분석

```
python3 _internal/scripts/analyze_latency.py --input <측정 디렉토리>  
python3 _internal/scripts/stats_poc_6_4_extended.py
```

⚠ `analyze_latency.py --input <dir>` 는 그 디렉토리에 cell별 `figures/paired_stats.csv` (cell 단위 회복률·효과크기)를 만든다 — 본 연구 결과를 cell 단위로 대조하기엔 이것으로 충분하다. 반면 `stats_poc_6_4_extended.py` (3 평면 통합 재집계)는 `phase2` · `phase3` · `phase4_extension` 의 `figures/paired_stats.csv` 경로를 **하드코딩**해 있으므로, 측정을 다른 디렉토리에 했다면 그 결과를 `phase4_extension/` 에 두거나 스크립트 상단의 `PHASE4_EXT_DIR` 상수를 측정 디렉토리로 바꿔야 통합 재집계에 반영된다.

6.5 하네스 없이 직접 재현하는 최소 예시

채림님이 한 쿼리를 4 조건으로 직접 돌려보고 싶을 때, psql에서:

```
-- 공통 준비
LOAD 'vector';
SET vector.update_sample_size = off;
SET plan_cache_mode = force_custom_plan;

-- ① baseline (기본엔진 - 개입 없음)
SET vector.disable_estimation = on;
EXPLAIN (ANALYZE, FORMAT JSON) <VAQ 쿼리>;

-- ② B1 / CaseB / oracle (카디널리티 주입)
SET vector.disable_estimation = off;
SET vector.injected_card = <카디널리티 값>; -- B1=est_b1 / CaseB=(est_b1+est_method)/2 / oracle=true_card
EXPLAIN (ANALYZE, FORMAT JSON) <VAQ 쿼리>;
```

⚠ Exqutor 주입은 실행 시점 2-pass라 순수 **EXPLAIN** (미실행)으로는 pass-1 플랜만 보인다 — pass-2 플랜·실제 latency를 보려면 **EXPLAIN ANALYZE** (실행 포함) 또는 실효리 직접 실행이 필요하다. **injected_card** 를 0 이상으로 줬는데 **Estimated cardinality** 로그가 안 뜨면 벡터 테이블이 IndexScan 경로로 풀린 것(주입 미발동) — **SET enable_indexscan = off** 등으로 SeqScan을 유도하거나 더 큰 sf로 확인한다.

★ 위 **EXPLAIN ANALYZE** 예시는 latency 관찰·주입 발동 확인용이다. **정확한 pass-2 실행 계획(plan_signature)**을 캡처하려면 하네스 **_capture_plan** 이 쓰는 방식이 필요하다 — **LOAD 'auto_explain'** 을 **LOAD 'vector'** 보다 **먼저** 로드 → **client_min_messages = log** · **auto_explain.log_min_duration = 0** · **auto_explain.log_format = json** 설정 → 실효리 실행 → client notice로 도착하는 pass-2 플랜(최대 duration 블록)을 파싱(\$4.1~\$4.2).

7. 실험 데이터 위치

모든 측정 raw·집계는 로컬 **_internal/cache/rq3/latency/** 에 있다 (서버 종료에 대비해 전량 회수 완료). 디렉토리 구조·JSON 스키마·CSV 설명은 그 안의 **README.md** 참조.

경로	내용
phase2/ · phase3/	DEEP sf=10 정본 측정 raw (20 cell)
phase4_extension/	SIFT·SSN·YFCC sf=10 측정 raw (36 cell) + 추정치 parquet
poc_6_4_extended/	3 평면 통합 집계 — summary.md + CSV 9종 (정본 수치)

8. 참고 자료

- 6/11 최종보고서: **submission/_drafts/속도는벡터_6_11_최종보고서_20260520_171521.{md,pdf}** — 연구 전체 맥락 (\$5 엔진 적용 검증)
- 측정 하네스: **_internal/scripts/measure_latency_realengine.py**
- 분석 스크립트: **_internal/scripts/analyze_latency.py** · **stats_poc_6_4_extended.py**
- Exqutor 논문: arXiv:2512.09695v2

작성: 2026-05-22 17:35 KST · 속도는벡터. 문의는 조현빈.